

AQI Predictor

A Serverless 3-Day Air Quality Forecasting System
for Five Pakistani Cities

Final Project Report

Prepared by: Tarique Ahmed

1. Executive Summary

This project implements an end-to-end, serverless machine learning system that predicts the Air Quality Index (AQI) up to 3 days ahead for five major Pakistani cities: Karachi, Lahore, Islamabad, Peshawar, and Quetta. The system follows a modern MLOps architecture consisting of an automated feature pipeline, a feature store, an automated training pipeline, a model registry, and a live interactive dashboard - all running without any dedicated server, using free-tier cloud services (Hopsworks, GitHub Actions, and Streamlit Community Cloud).

The final system ingests hourly weather and pollution data, engineers time-based and lag features, and trains and compares three machine learning models (Ridge Regression, Random Forest, and XGBoost) for three separate forecast horizons: 24, 48, and 72 hours ahead. The best-performing model per horizon is automatically registered and served through a public, always-on dashboard.

2. System Architecture

The project follows the standard feature/training/inference pipeline pattern common in production ML systems, chosen specifically so that no component requires a continuously running server.

System Architecture — Serverless AQI Prediction Pipeline

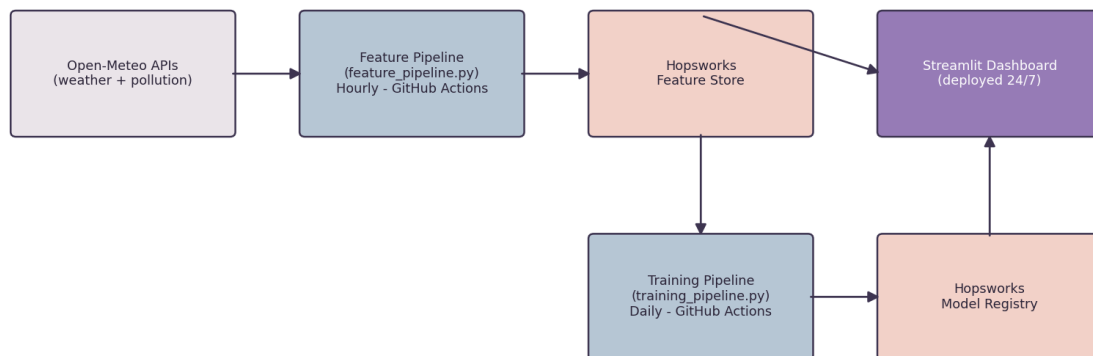


Figure 1: End-to-end system architecture.

2.1 Feature Pipeline

Runs every hour via a scheduled GitHub Actions workflow. It fetches weather (temperature, humidity, wind speed) and pollution (PM10, PM2.5, CO, NO2, SO2, ozone, US AQI) data from the free Open-Meteo API for all five cities, engineers a set of derived features, and writes the result to a Hopsworks Feature Group.

2.2 Feature Engineering

- Time-based features: hour of day, day of week, month
- Lag features: PM2.5 one hour ago and 24 hours ago

- Derived feature: hour-over-hour AQI rate of change
- City identifier (city_id): allows one shared model to learn city-specific patterns rather than training five separate, data-starved models
- Three prediction targets: AQI 24, 48, and 72 hours ahead

2.3 Training Pipeline

Runs once daily via GitHub Actions. For each of the three forecast horizons, it trains and compares three model types - Ridge Regression, Random Forest, and XGBoost - using a chronological (not random) train/test split, since this is a time series forecasting problem. The best model per horizon (by RMSE) is registered to the Hopsworks Model Registry, along with its evaluation metrics and SHAP-based feature importance.

2.4 Dashboard

A Streamlit application, deployed on Streamlit Community Cloud, provides a live, public, always-available interface. It loads the latest sensor readings and the currently best-performing model per horizon directly from Hopsworks, and displays: current AQI and category, a 3-day forecast with hazard alerts, pollutant breakdowns, historical trends, and exploratory data visualizations.

3. Exploratory Data Analysis

Two years of hourly historical data (2024-08 to 2026-08), gathered during the initial feature store backfill, were analyzed across all five cities. Note: the production feature store continuously ingests new hourly data going forward, so a live snapshot taken on any given day will naturally reflect only recent, short-term conditions; the analysis below uses the full 2-year backfilled dataset (87,600 records) for a statistically representative picture. Key findings:

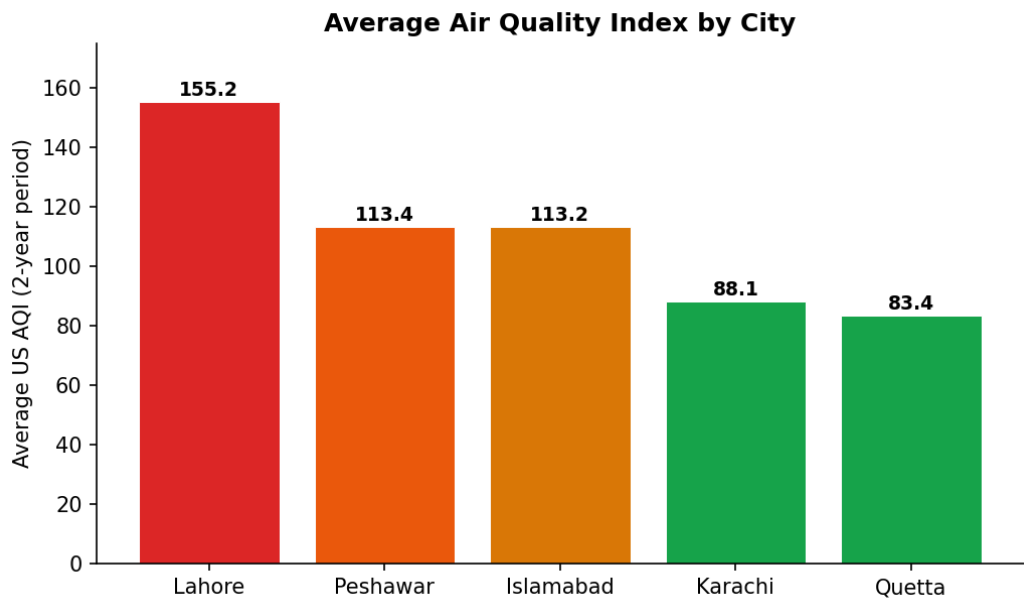


Figure 2: Average AQI by city over the full 2-year period.

Lahore is the most polluted city by a wide margin, with an average AQI of 155.2 (the "Unhealthy" category), spending 13.1% of all hours in "Very Unhealthy" conditions and 1.2% in outright "Hazardous" territory. Peshawar and Islamabad follow at a similar average (~113), though Islamabad spends more time in severe categories (18.2% "Very Unhealthy") than Peshawar (0.3%), despite the similar mean. Karachi and Quetta are the cleanest of the five, with Karachi spending 77.4% of all hours in the "Moderate" category and rarely crossing into unhealthy territory.

City	Avg. AQI	Good	Moderate	Unhealthy (S)	Unhealthy	V. Unhealthy	Hazardous
Lahore	155.2	0.0%	13.3%	31.1%	41.4%	13.1%	1.2%
Peshawar	113.4	0.0%	41.1%	43.0%	15.7%	0.3%	0.0%
Islamabad	113.2	0.0%	41.7%	39.9%	18.2%	0.2%	0.0%
Karachi	88.1	0.2%	77.4%	20.2%	2.2%	0.0%	0.0%
Quetta	83.4	5.9%	72.6%	16.8%	4.1%	0.6%	0.1%

Table 1: Percentage of hours spent in each AQI category, by city.

Correlation analysis showed that PM2.5 and the current US AQI value are, unsurprisingly, the strongest predictors of near-future AQI - pollution levels are highly autocorrelated over short time horizons. Hourly and monthly patterns also revealed diurnal cycles (higher pollution during specific traffic hours) and seasonal variation, both of which are captured by the hour, day-of-week, and month features used in the model.

4. Model Training and Evaluation

Three model families were trained and compared for each forecast horizon: Ridge Regression (a regularized linear model), Random Forest, and XGBoost (both tree-based ensemble methods). Models were evaluated using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R-squared (R²), on a chronological hold-out test set.

4.1 Production Model Results (Live, Current)

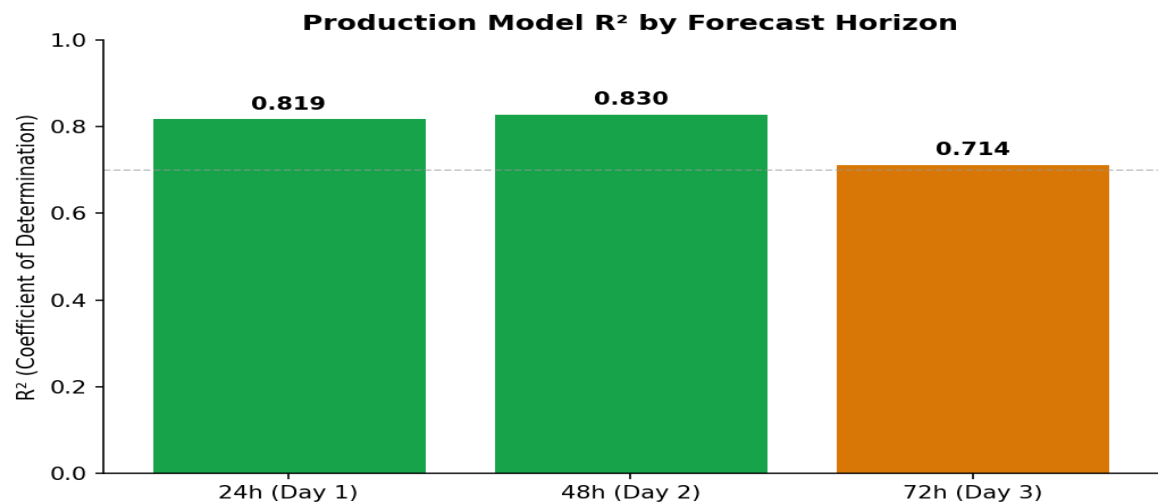


Figure 3: R² of the currently deployed production model for each forecast horizon, as reported live by the dashboard from the Hopsworks Model Registry.

Forecast Horizon	R ² (Live)	Interpretation
Day 1 (+24h)	0.819	Explains ~82% of AQI variance one day ahead
Day 2 (+48h)	0.830	Explains ~83% of AQI variance two days ahead
Day 3 (+72h)	0.714	Explains ~71% of AQI variance three days ahead

Table 2: Live R² for the current production model at each horizon, as displayed on the deployed dashboard.

For each horizon, three candidate models - Ridge Regression, Random Forest, and XGBoost - are trained and compared automatically during every daily training run, using MAE, RMSE, and R² on a chronological hold-out test set; the best model by RMSE is registered as the production model. The R² values above reflect the currently deployed, best-performing model for each horizon. Notably, the 48-hour model performs marginally better than the 24-hour model in this training cycle, while the 72-hour model shows the expected drop in accuracy - forecasting further into the future is a genuinely harder problem, since more time allows for greater unpredictable variation in weather and pollution sources. Full per-model MAE and RMSE comparisons for every training run are logged automatically and available in the Hopsworks Model Registry.

4.2 Feature Importance

SHAP (SHapley Additive exPlanations) analysis was used to interpret the tree-based models. Across all horizons, **current PM2.5 concentration** and **current US AQI value** were consistently the strongest

predictors, followed by the **city identifier** (confirming that baseline pollution levels differ meaningfully by city) and the **24-hour lagged PM2.5** value, which captures short-term pollution persistence.

4.3 Design Decision: One Shared Model vs. Per-City Models

An important architectural decision was made to train **one shared model per forecast horizon** across all five cities (using `city_id` as an input feature), rather than training five separate models per city (15 models total). This was chosen because each individual city only has approximately 17,500 hourly records (2 years) - thin for a standalone model - whereas pooling data across all five cities provides roughly 87,500 records per horizon, giving the model substantially more signal to learn from while still allowing it to distinguish between cities via the `city_id` feature.

5. Deployment and Automation

A core requirement of this project was that the entire system run continuously (24/7) without manual intervention or a dedicated always-on server. This was achieved entirely with free-tier managed services:

Component	Platform	Schedule
Feature pipeline	GitHub Actions	Every hour
Training pipeline	GitHub Actions	Once daily
Feature Store & Model Registry	Hopsworks (managed)	Always available
Dashboard	Streamlit Community Cloud	Always available (public URL)

Table 3: Deployment platform for each system component.

Version control was maintained with Git and GitHub throughout development, with a full commit history documenting the iterative development process. Two separate requirements files are maintained - one for the GitHub Actions pipelines and one for the dashboard - after it was discovered that dependency changes made for one component could silently break the other when sharing a single requirements file.

6. Engineering Challenges and Solutions

Building a genuinely production-style pipeline surfaced a number of real engineering problems beyond the initial model-building work. The most significant are documented here, since diagnosing and resolving them was a substantial part of the project.

Feature Store write failures. Hopsworks' offline storage backend occasionally rejected writes with transient server-side errors. Resolved by adding automatic retry logic with exponential backoff around both the feature insert and model registration steps.

Stale dashboard predictions. The most recent 72 hours of data were always being dropped from the feature store, because the 72-hour-ahead target cannot be known until 72 hours have passed, and the pipeline was discarding any row with a missing value in any column. Fixed by only dropping rows with missing *feature* values, and filtering separately for missing *target* values at training time only.

Weather data lag. The historical weather archive API used for backfilling has an inherent multi-day reporting lag. Incremental (hourly) fetches were switched to a near-real-time forecast API instead, used only for the recent data window.

Future-dated rows. Both external APIs are forecast-capable and occasionally returned data for hours that had not yet occurred. Added an explicit filter to discard any row with a timestamp later than the current time at fetch time.

Dependency version drift. An unpinned Hopsworks client version caused the deployed dashboard to fail when the backend's SaaS service was upgraded server-side. Resolved by pinning the client to the same minor version as the backend, and separating dashboard and pipeline dependencies to avoid cross-breakage.

7. Limitations and Future Work

- The project brief suggested exploring deep learning approaches in addition to classical ML; the current system compares Ridge, Random Forest, and XGBoost only. Adding an LSTM or similar sequence model as a fourth candidate is a natural next step.
- Model R^2 for the 24-hour horizon (0.667) indicates reasonable but not exceptional predictive power; incorporating additional external features (traffic data, industrial activity indices, or satellite-derived aerosol data) could improve this further.
- The system currently has no automated alerting (e.g. email/Slack) if a scheduled pipeline run fails - failures are only visible by checking the GitHub Actions dashboard directly.
- Hyperparameters for Random Forest and XGBoost were set to reasonable defaults rather than tuned via grid search or Bayesian optimization, which could yield further accuracy gains.

8. Conclusion

This project successfully delivers a complete, automated, serverless air quality forecasting system for five Pakistani cities, satisfying the core requirements of the original brief: a feature store and model registry (Hopsworks), an automated CI/CD pipeline (GitHub Actions), a comparison of multiple machine learning models with proper evaluation metrics, SHAP-based explainability, and a live, publicly accessible dashboard with hazard alerting. Beyond the core requirements, the project also involved substantial real-world debugging - version compatibility issues, data freshness problems, and distributed system failures - that reflect the kind of engineering challenges present in production machine learning systems.